Week 10 Google ADK Agent Programming — Hands-on Lab Code by Section

`10-12주차강의노트(Google ADK Agent Programming)_20261111f.pptx` ... into folder-based projects. Installation, project structure, `.env`, and `adk run/web` follow the adk.dev Python Quickstart. Verified on **google-adk 2.6.3** (Python 3.11). The Appendix A labs (`appendixA_python_for_adk/`) and Part 12 observability labs (`part12_observability/`) were verified in this repository's `.venv` (**google-adk 2.14.0** · Python 3.12).

> 💡 **Reference Guides**:
>
> - API Key Issuance and Quota Guide: AI Studio_API_Key.md
> - Frequently Occurring Errors and Solutions: TROUBLESHOOTING-KR.md

Development Environment Setup (Using `uv`)

These labs are based on uv, an extremely fast Python package and virtual environment manager.

1. Install `uv`

Windows

- **PowerShell (Officially Recommended)**:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- **winget**:

```
winget install --id=astral-sh.uv -e
```

- **Using pip**:

```
pip install uv
```

Linux / macOS

- **curl (Officially Recommended)**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **Using pip**:

```
pip install uv
```

> After installation, restart your terminal window or reload your environment variables and verify that `uv --version` works properly.

2. Install Python 3.11+ and Create a Virtual Environment

ADK 2.x requires Python 3.11 or higher. Even if the required Python version is not installed on your system, `uv` can automatically download it and create a virtual environment.

```
# Install Python 3.12 (if needed)
uv python install 3.12

# Create a virtual environment based on Python 3.12
uv venv --python 3.12
```

3. Activate the Virtual Environment

- **Windows (PowerShell)**:

```
.venv\Scripts\Activate.ps1
```

`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`,

- **Windows (CMD)**:

```
.venv\Scripts\activate.bat
```

- **Linux / macOS**:

```
source .venv/bin/activate
```

4. Install Packages

```
uv pip install -r requirements.txt
```

> - `google-agents-cli` (`part13_agents_cli`) ,
> - 💡 **Tip (Using `uv run`)**: You can run commands directly without activating the virtual `uv run agents-cli --help`,

(Optional) Install Google Cloud CLI (`gcloud`)

If you plan to use Vertex AI integration (Method ②) or deploy to Cloud Run (part11 · part12), you need the `gcloud` CLI installed on your system. (`gcloud` is an OS-installed binary tool, not a Python pip package.)

- **Windows**:

```
winget install Google.CloudSDK
```

(Or download the official installer)

- **Linux / macOS**:

```
curl https://sdk.cloud.google.com | bash
exec -l $SHELL
```

- **Account Authentication after Installation (ADC)**:

```
gcloud auth application-default login
```

5. Configure Environment Variables (`.env`)

Copy `.env.example` to `.env` and configure **one of the following authentication methods**. Since `adk` searches upward from the agent folder for `.env`, placing a single `.env` file at the root (`ADK_Labs/.env`) applies across all Parts.

Method ①: Google AI Studio (API Key Method, Recommended by Default)

- Get an API key from Google AI Studio and enter it:

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=your-gemini-api-key
```

- For detailed quota and Free/Paid tier information, see AI Studio_API_Key.md.

Method ②: Google Cloud Vertex AI (GCP Project Integration)

If you want to use Google Cloud credits or run high-volume tests without the per-minute rate limits (429) of a free API key, you can switch to Vertex AI.

1. **Prepare a GCP Project and Enable the API**:
   - Create a project in the Google Cloud Console
   - Enable the **Vertex AI API** (`aiplatform.googleapis.com`)
2. **Authenticate with Google Cloud SDK (`gcloud`) (Application Default Credentials)**:
   - Run the following command in your local terminal to log in with your Google account:

```
gcloud auth application-default login
```

3. **Configure `.env`** :
   - Set `GOOGLE_GENAI_USE_VERTEXAI=TRUE` and enter your project ID and location (comment out `GOOGLE_API_KEY`):

```
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_CLOUD_LOCATION=asia-northeast3   # Seoul region (or us-central1)
# GOOGLE_API_KEY= (commented out)
```

> ⚠ **MCP Compatibility Note**: The MCP example (`part06_tools`) requires `mcp` 1.x. If `mcp` 2.x is
> `uv pip install "mcp>=1.24,<2"`

Execution Rules

- Each `partNN_…/` folder (`NN` = lecture note Part number) is a **parent directory of agents**. Inside it, `xxx_agent/agent.py` defines `root_agent` .
  - Run interactively: `cd partNN_…` then `adk web --port 8008` or `adk run <agent_folder>`
  - Run via script: `cd partNN_…` then `uv run run_*.py`
- `appendixA_python_for_adk/` contains the Appendix A (Python for ADK) labs. Instead of agent folders, it runs standalone scripts `a1_…py` through `a7_…py` : `cd appendixA_python_for_adk` then `uv run run_all.py --quiet`
- 🟢 = Runs without an LLM (no API key required). 🔑 = Requires a Gemini API key.

Lecture Note Sections ↔ Lab Folder Map

The lab folder numbers match the **Part numbers** of the lecture notes (`_20261111f.pptx` , 198 slides): `partNN_…` = `Part N` in the Table of Contents (Slide 3) and section cover slides. Appendix A is in `appendixA_python_for_adk/` . References to "Lab partNN …" on the slides also point to these folders. The Part names, subtitles, and slide ranges in the table below match the Table of Contents and section cover slides verbatim. The "▸ Appendix A" chip in the upper-right corner of main slides jumps to the related appendix slide, whose hands-on code can be found in the map in `appendixA_python_for_adk/README-KR.md` .

Lecture Note Part (Cover Title · Subtitle)	Slides	Lab Folder	Lab Contents (Parentheses = Slide Numbers)	Scripts
Part 1 · 01. Framework Landscape and ADK Comparison of Top 5 Frameworks, Architectural Paradigms, Selection Criteria	5–15	`part01_frameworks/`	Concept lecture (no lab code; folder contains only a guide README)	—
Part 2 · 02. Google ADK Architecture The Internal Engine Seen Through Runner · Event · Session · Memory	16–29	`part02_architecture/`	`adk-based_Agent/` : `chat_time_weather_agent` (real-time time, weather, chat) · `memory_layers_agent` (dual-layer memory, 29) — `Direct_ADK/` : Event lifecycle (21–28) runner and direct Runner examples	In `Direct_A` `run_workfl` `run_memory`
Part 3 · 03. Development Environment and CLI adk create · run · web · api_server and agents-cli	30–48	`part03_cli/`	`my_agent` (`adk create` structure) · Direct Runner code (23–25, 39) · `adk` CLI source structure & call path (32–36, `adk_cli_map.py`) · Reproducing `adk` internals (37–38) · `adk` commands (40–43, `api_server` 43) · `agents-cli` overview (44–48) is in `part13_agents_cli/`	`README-KR.` 🟢/🔑 `adk` `adk_cli_ma`
Part 4 · 04. LLM Agent Programming Agent Parameters, Instructions, Structured Output, Models, Runner	49–56	`part04_llm_agent/`	weather (50) · tutor (State template, 52) · reviewer (`output_schema` , 53) · litellm (54) · planner (55) · Runner (56)	`run_wit` `[weather|tu`
Part 5 · 05. Skills for ADK Agents SKILL.md · Progressive Disclosure · SkillToolset · google.adk.skills · Agents CLI Skills	57–74	`part05_skills/`	`skills_agent` (SkillToolset, 63–67) · `skills_explorer.py` (`google.adk.skills` loading, validation, tool exposure, 64–67 · coding agent skills 62, 72) · Installing coding agent skills (70–73) is in `part13_agents_cli/coding_with_ai/`	`skills_` `[load|valid` `run_skills`
Part 6 · 06. Tools: The Hands and Feet of an Agent Function · Built-in Tools · AgentTool · MCP · OpenAPI · Long-Running	75–82	`part06_tools/`	shop (Function Tool + confirmation, 77) · analyst (AgentTool · `google_search` · code execution, 79) · mcp (80) · openapi (HR API + mock server, 81) · refund (LongRunning, 82)	`run_too` `[shop|analy`
Part 7 · 07. Session · State · Memory · Callback How Agents Remember, Pass State, and Monitor Themselves	83–89	`part07_context/`	`state_agent` (prefixes · SQLite session, 85–86) · `memory_agent` (87) · `guarded_agent` (callbacks, 88–89) · `AuditPlugin` (89)	🔑 `run_sta` `run_memory`
Part 8 · 08. Multi-Agent Composition sub_agents Delegation, 1.x Workflow Agents, 2.0 Collaboration Modes	90–95	`part08_multi_agent/`	`help_desk` (`sub_agents` transfer, 92) · `report_pipeline` (1.x Sequential/Parallel/Loop, 93) · `travel_planner` (2.0 coordinator, 94–95)	`run_mul` `[help_desk|`
Part 9 · 09. ADK 2.0 Graph Workflow Pinning Down Control Flow in Code with Node · Edge · Route · JoinNode	96–105	`part09_graph_workflow/`	seq (100) · route (101) · fan (JoinNode, 102) · loop (back-edge, 103) · refund (Case ①, 104) · expense (Case ②, 105)	🟢 `run_wor`
Part 10 · 10. Dynamic · HITL · Retry Orchestrating in Code, Waiting for Humans, and Surviving Failures	106–114	`part10_dynamic_hitl/`	dynamic (107) · dynamic_parallel (108) · hitl · feedback (110) · approval_gate (111) · retry (112)	🟢 `run_par`
Part 11 · 11. Evaluation · Deployment · Operations adk eval, Cloud Run, Agent Runtime, Triggers, A2A, Observability	115–129	`part11_eval_deploy/`	evalset · `AgentEvaluator` (116) · `adk gcloud run deploy` , Runtime deployment paths via `adk` / `agents-cli` /SDK (126) · Pub/Sub trigger (127) · A2A (128)	`make_evals` `test_conta` `run_server` `deploy/dep` `DRY_RUN=1` 🔑 `run_a2a`
Part 12 · 12. Observability Understanding the Agent's 'Why' with Trace · Metrics · Logs — OpenTelemetry · OpenMetrics · OpenInference	130–142	`part12_observability/`	Span tree · schema v1/v2 (134) · GenAI attributes (135) · Built-in metrics · `adk web --otel_to_cloud` (137) · Content logging and privacy (138) · OpenMetrics `/metrics` (139) · OpenInference · Phoenix (140)	🟢 `run_all` obs5_otlp_e obs6_openir web`)
Part 13 · 13. agents-cli End-to-End Workflow Anatomy of a Real Project: create · playground · eval · deploy · run	143–156	`part13_agents_cli/`	Project anatomy (144) · 6-stage development lifecycle (145) · Coding with AI (146–147, `coding_with_ai/`) · One server, two entrypoints (148–149) · eval (150–151) · deploy · infra (152–153) · OpenTelemetry pipeline (154, `telemetry/`) · Invocation (155) · Lab (156)	`adk|a2a`
Part 14 · 14. Capstone Lab & Synthesis Assembling Today's Building Blocks into a Single Agent	157–162	`part14_lab/`	Academic inquiry agent Steps 1–6 (158): Basic → Tools → Guardrails → Route Workflow → HITL+Retry → eval/deploy · Step 7 Observability (159, `step7_observability/`)	`run_lab.py` 🟢)
Appendix A · Appendix A. Python for ADK async/await · Event Loop · Concurrency · Type Hints · Object Model → Runner	163–198	`appendixA_python_for_adk/`	A1 Async & `asyncio` (165–178) · A2 Concurrency control (179–181) · A3 Generators & streaming (182–185) · A4 `async for` & `with` (186–187) · A5 Type hints & Pydantic (188–190) · A6 Object model & extension (191–194) · A7 Runner (195–197) — item-by-item map in README	🟢 `run_all` `a7_runner.`

`common/` contains shared utilities, providing `.env` loading and a Runner wrapper (`Session.send` / `Session.resume_input` : resuming HITL).

Commonly Used Commands

```
cd part09_graph_workflow && adk web --port 8000
```

```
cd part11_eval_deploy && adk eval weather_agent weather_agent/weather.evalset.json --config_file_path
weather_agent/test_config.json --print_detailed_results
```

How to Run `adk eval`

`Eval module is not installed` 、 、 、 `D:\Python311\Scripts\adk.exe` 、 `C:\Users\...\Python311\python.exe`). In that case, run `python -m google.adk.cli` or `uv run adk` instead of `adk` . The arguments remain the same.

Evaluating the Part 11 weather agent (from the `part11_eval_deploy` folder):

```
adk eval weather_agent weather_agent/weather.evalset.json --config_file_path
weather_agent/test_config.json --print_detailed_results
# Or: python -m google.adk.cli eval weather_agent weather_agent/weather.evalset.json --config_file_path
weather_agent/test_config.json --print_detailed_results
```

Evaluating the `part14_lab` academic inquiry agent (from the `part14_lab` folder, example output: `Tests passed: 3`):

```
adk eval step3_callback step6_eval_deploy/academic.evalset.json --config_file_path
step6_eval_deploy/test_config.json
# Or: python -m google.adk.cli eval step3_callback step6_eval_deploy/academic.evalset.json --
config_file_path step6_eval_deploy/test_config.json
```

Evaluation requires the extra packages from `google-adk[eval]` (e.g., `rouge-score`), which are installed automatically via `uv pip install -r requirements.txt` . `adk web` , `adk run` , and `adk api_server` `python -m google.adk.cli web --port 8000` ,

```
cd part11_eval_deploy/a2a_demo && uv run uvicorn prime_agent.agent:a2a_app --port 8001
```

```
cd part06_tools && uv run openapi_agent/hr_mock_server.py
```

How HITL Resumption Works (part09 · part10 · part14)

When `RequestInput` executes, the workflow emits an `adk_request_input` `function_call` event and pauses. To resume, send a `FunctionResponse(name="adk_request_input", response=...)` with the same `interrupt_id` and `invocation_id`. `Session.resume_input()` in `common/runner_util.py` handles this process automatically. In `adk web`, you can respond directly in the UI input box.

Notes on Using a Free API Key

- The Gemini Free Tier is limited to **roughly 5 requests per minute and 20 requests per day per model**. A single agent turn typically makes 2–4 LLM calls.
- `common/bootstrap.py` automatically retries 429 and 5xx responses at 10s → 20s → 40s intervals (applies only when running `run_*.py`; set `ADK_LAB_RETRY=0` to disable). If you exceed the daily quota, retries will not help—switch `ADK_MODEL` in `.env` to a different model.
- `google_search` (`searcher` in `part06` analyst) was blocked with a 429 error on the free key. Run it with a billing-enabled API key or Vertex AI.
- `"Eval module is not installed"`, `python -m google.adk.cli eval ...`, `adk web` `adk run` can be run the same way).

Verification Results (google-adk 2.6.3)

Results when running without an API key:

- 🟢-marked scripts (Steps 4 and 5 of `part02` · `part09` · `part10` · `part14`, and guardrail blocking in `part07`) match the expected outputs on the slides. `part02` `run_memory_layers.py offline`: only `user:name` persists in a new session while `trip` is absent; `InMemory` sessions vanish on restart whereas SQLite sessions persist; `search_memory('부산')` returns 1 match and `'부산에서'` returns 0 matches. Examples: `auto-approved $42`, `joined={...}`, `sum of squares=14`, `3회차 성공` (succeeded on 3rd attempt).
- All 38 agent packages import cleanly and create `root_agent` properly.
- 🟢 `part05_skills/skills_explorer.py`: L1 listing · L2/L3 loading · `<available_skills>` XML, 4 invalid skills fail at load time, `get_weather` is exposed after activation, and 7 coding agent skills (`google-agents-cli-*`) are loaded via the ADK loader.
- 🟢 `part11_eval_deploy/tests/test_container_app.py`: the container server for `gcloud` deployment (`main.py`) lists `weather_agent` on `/list-apps` and stores sessions in the `SESSION_SERVICE_URI` DB (2 passed). `deploy_agent_runtime_sdk.py` (dry-run): creates `AdkApp` and prints SDK deployment configuration.
- 🟢 `part11_eval_deploy/run_server_state.py`: in-memory sessions are lost across instance changes while shared DB sessions persist; selects `InMemorySessionService` when Cloud Run (`K_SERVICE`) is detected.
- Re-verified on 2026-11-12 (this repository's `.venv`, `google-adk 2.14.0` · Python 3.12, with LLM calls blocked): 44 agent packages imported and `root_agent` created OK; 16 🟢 scripts (`part02` · `03` · `05` · `07` · `09` · `10` · `11` · `14`) exited cleanly; `part11` `tests/test_container_app.py` 2 passed; `part13` `tests/unit` 6 passed. Slide numbers in each folder's README and code comments correspond to `_20261111f.pptx` (198 slides).
- 🟢 `part12_observability/run_all.py` (`google-adk 2.14.0` · `opentelemetry-sdk 1.45.2`, `invoke_workflow > invoke_agent > generate_content · execute_tool`, under `SPAN_ONLY`, metrics → OpenMetrics `/metrics` response verified. With optional packages installed, `obs5` (OTLP reception verified) and `obs6` (OpenInference spans & default content recording) also pass OK. `part14_lab/run_lab.py 7`: 6 requests = 6 traces, 3 retries appear as `invoke_node fetch_registrar`, and no PII leakage.
- 🟢 `appendixA_python_for_adk/run_all.py` (`google-adk 2.14.0` · Python 3.12): all 7 scripts OK. Scheduling order matches the table on Slide 175, partial events are not saved to the session, `run_async` produces at the caller's pace while `run()` produces ahead in the background, and 2 tools in a single response execute concurrently. For differences from 2.6.3, see 'Version Differences' in the Appendix README.

`gemini-3.5-flash-lite`,

Lab Folder	Result
part02	`get_current_time` call → response, 4 events recorded · `memory_layers_agent` : in Session 2, recalls name via `user:` State and "cannot eat spicy food" via `preload_memory`
part03	`run_direct_runner.py` : `function_call` → `function_response` → final response · `adk_internals.py` : default session = `my_agent/.adk/session.db` (`PerAgentDatabaseSessionService`), `--in_memory` → `InMemory` , `--ask` single-turn response
part04	`weather` · `tutor` (State template) · `reviewer` (`state['review']` structured output) · `planner` (thought output) · `litellm` (Gemini fallback) all working
part05	`skills` (`list_skills` → `load_skill` → `get_weather` → `run_skill_script` → `load_skill_resource` , heatwave & lookup-failure exception rules applied) working
part06	`shop` (payment after `adk_request_confirmation` approval) · `calculator` code execution · MCP (`mcp` 1.30 + Node) · OpenAPI (mock server) · `refund` (LongRunning resumption) working. **Only `google_search` failed due to free key quota limits**
part07	`user:` / `app:` prefixes persist across new sessions · Memory recall · PII blocking / quantity validation / `AuditPlugin`
part08	`billing` → `tech` transfer · Parallel → draft → Loop · `single_turn` weather + `task` flight booking (result `BK-KE1201-W`)
part09	All workflows and LLM nodes (refund notice, expense review, `seq`) working
part11	`pytest tests` 2 passed · A2A remote `prime_agent` delegation working
part13	`agents-cli eval run`
part14	Steps 1–5 working · `adk eval` academic 3/3 passed